IMAGE PROCESSING LABORATORY

Social Media Data Analysis - A.A. 2023-2024

Sentiment Analysis and Recommendation Systems

Francesco Ragusa https://iplab.dmi.unict.it/ragusa/ (https://iplab.dmi.unict.it/ragusa/) - francesco.ragusa@unict.it (mailto:francesco.ragusa@unict.it)

In this laboratory we will see how to:

- Build a recommendation system based on collaborative filtering;
- Use word embeddings using Spacy;
- Perform sentiment analysis with VADER;

1. Recommendation Systems

1.1 Dataset

We will use a dataset movies rated by users. We can load the data directly from its URL as follows:

In [2]:
```
import pandas as pd
data = pd.read_csv('https://iplab.dmi.unict.it/ragusa/downloads/movieratings.c
sv')
data.info()
data.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100003 entries, 0 to 100002
Data columns (total 4 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
 0   user_id     100003 non-null  int64
 1   item_id     100003 non-null  int64
 2   rating      100003 non-null  int64
 3   item_title  100003 non-null  object
dtypes: int64(3), object(1)
memory usage: 3.1+ MB
```

Out[2]:

	user_id	item_id	rating	item_title
0	0	50	5	Star Wars (1977)
1	290	50	5	Star Wars (1977)
2	79	50	4	Star Wars (1977)
3	2	50	5	Star Wars (1977)
4	8	50	5	Star Wars (1977)

The dataset contains 100003 observations. Each observation consists in a rating given by a user to a movie. Specifically, the variables have the following meaning:

- `user_id` the id of the user rating an item;
- `item_id` the id of the item (a movie);
- `rating` a number between 1 and 5 indicanting the rating given by the user to the movie;
- `item_title` : the title of the rated movie.

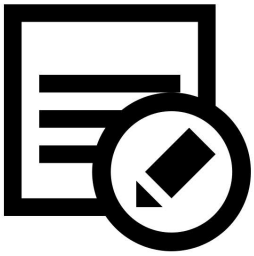
Each user can rate an item at most 1 time, hence each line can be related to different users rating the same item or to different items rated by the same user. Let's count how many unique users and items we have in the dataset:

In [3]:
```
print("Number of users: {}".format(data['user_id'].nunique()))
print("Number of items: {}".format(data['item_id'].nunique()))
```

```
Number of users: 944
Number of items: 1682
```

Question 1

How many rows we would have in the dataset if each user had rated each item? If we constructed a utility matrix, would it be sparse of dense?



Answer 1

Before proceeding, let's split the dataset into a training and a test set:

In [4]:
```
import numpy as np
import random
from sklearn.model_selection import train_test_split
np.random.seed(42)
random.seed(42)
train_data, test_data = train_test_split(data, test_size=0.25, )
```

Let's now build the utility matrix. Recall that it is a matrix showing how a user has rated a given item. We can create the utility matrix using a pivot table:

In [5]:
```
U = train_data.pivot_table(index='user_id',columns='item_id',values='rating')
U.head()
```

Out[5]:

item_id	1	2	3	4	5	6	7	8	9	10	...	1673	1674	1675	1676	1
user_id																
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	N
1	5.0	3.0	4.0	3.0	3.0	5.0	NaN	NaN	5.0	3.0	...	NaN	NaN	NaN	NaN	N
2	4.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2.0	...	NaN	NaN	NaN	NaN	N
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	N
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	N

5 rows × 1638 columns

The matrix is sparse. It contains `NaN` values for user-movies pairs for which we do not have a rating.

1.2 Collaborative Filtering

We will now implement a user-user collaborative filter. Remember that a user-user collaborative filter works as follows:

1. Consider a user x_i and an item s_j which not been rated by user x_i;
2. Build a profile for each user by considering the rows of the utility matrix normalized by subtracting the mean;
3. Find a set N of similar users who have rated item s_j;
4. Estimate the utility value $u(x_i, s_j)$ computing a weighted average of rating given by the similar users;

Let's first see an example for `N=3` , `x_i=0` and `s_j=1` :

In [6]:
```
N=3
xi = 0
sj = 1
```

First, we need to compute user profiles for all users. We can do this by first replacing all missing values with zeros, then normalizing each row by subtracting its mean (computed before adding the zeros):

In [7]:
```
profiles = (U-U.mean(1)).fillna(0)
profiles.head()
```

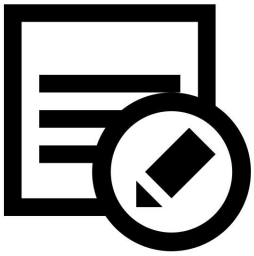
Out[7]:

user_id	0	1	2	3	4	5	6	7	8	9	...
0	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0	0.000000	...
1	0.0	1.300518	-0.891304	1.285714	-1.315789	0.253968	1.426829	0.0	0.0	0.722222	...
2	0.0	0.300518	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0	0.000000	...
3	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0	0.000000	...
4	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0	0.000000	...

5 rows × 1642 columns

Question 2

Why do we need to subtract the mean to obtain user profiles?



Answer 2

Now we should compute the cosine similarity between the row corresponding to x_i and all the other rows:

In [8]:
```
import numpy as np
cosine = lambda x,y: np.dot(x, y) / (np.sqrt(np.dot(x,x)) * np.sqrt(np.dot(y,
y)))

profiles.apply(lambda x: cosine(profiles.loc[xi],x), axis=1).head()
```

Out[8]:
```
user_id
0    1.000000
1    0.004347
2    0.087988
3    0.000000
4    0.116171
dtype: float64
```

The jth row reports the similarity distance between the jth profile and the profile of user x_i. The largest similarity (1) is between x_i and x_i. Let's remove this row:

In [9]:
```
similarities = profiles.apply(lambda x: cosine(profiles.loc[xi],x), axis=1).dr
op(xi)
similarities.head()
```

Out[9]:
```
user_id
1    0.004347
2    0.087988
3    0.000000
4    0.116171
5    0.000000
dtype: float64
```

We should now select only the users who have rated item s_j:

In [10]:
```
selected_users = U.loc[:,sj].dropna().index
selected_users
```

Out[10]:
```
Int64Index([  1,   2,   5,   6,  10,  13,  17,  20,  21,  25,
            ...
            923, 924, 927, 929, 930, 932, 933, 936, 938, 941],
           dtype='int64', name='user_id', length=355)
```

Question 3

Why do we need to select the users who have rated the considering item?

Answer 3

Let's now select the distances related to the selected users:

In [11]:
```
similarities.loc[selected_users].head()
```

Out[11]:
```
user_id
1     0.004347
2     0.087988
5     0.000000
6     0.011841
10   -0.108046
dtype: float64
```

We can now sort the distances with `sort_values` :

The last rows represent the most similar items. Let's select the last two items:

In [12]:
```
selected_similarities = similarities.loc[selected_users].sort_values()[-N:]
selected_similarities
```

Out[12]:
```
user_id
395    0.207021
679    0.214723
97     0.345497
dtype: float64
```

We can now store the user ids in an array:

In [13]:
```
similar_users = selected_similarities.index
similar_users
```

Out[13]:
```
Int64Index([395, 679, 97], dtype='int64', name='user_id')
```

Let's now see how the most similar users have rated movie s_j:

In [14]:
```
print(U.loc[similar_users[0],sj], U.loc[similar_users[1],sj], U.loc[similar_us
ers[2],sj])
```

```
5.0 3.0 4.0
```

We can now compute the average rating using a weighted average:

In [15]:
```
predicted_rating = (U.loc[similar_users,sj]*selected_similarities).sum()/selec
ted_similarities.sum()
predicted_rating
```

Out[15]:
```
3.9899620761918646
```

For convenience, let's now write an object to perform recommendations following the `scikit-learn` API:

In [16]:
```
class CollaborativeFilter():
    def __init__(self, N):
        self.N = N
        self.cosine = lambda x,y: np.dot(x, y) / (np.sqrt(np.dot(x,x)) * np.sq
rt(np.dot(y,y)))

    def fit(self, U):
        self.U = U
        self.profiles = (U-U.mean(1)).fillna(0)

    def predict(self, xi, sj):
        similarities = self.profiles.apply(lambda x: self.cosine(self.profile
s.loc[xi],x), axis=1).drop(xi)
        selected_users = self.U.loc[:,sj].dropna().index
        selected_similarities = similarities.loc[selected_users].sort_values()
[-self.N:]
        similar_users = selected_similarities.index
        predicted_rating = (self.U.loc[similar_users,sj]*selected_similaritie
s).sum()/selected_similarities.sum()
        return predicted_rating
```

We can use the Collaborative Filter as follows:

In [17]:
```
cf = CollaborativeFilter(N=3)
cf.fit(U)
print(cf.predict(xi,sj))
print(cf.predict(5,18))
```

```
3.9899620761918646
4.125945120766068
```

1.3 Performance Assessment

To assess the performance of our method, we can see how the filter works with the test data, for which we have the ground truth ratings:

In [18]:
```
test_data.head()
```

Out[18]:

	user_id	item_id	rating	item_title
87184	510	326	4	G.I. Jane (1997)
98871	151	1264	4	Nothing to Lose (1994)
1956	1	265	4	Hunt for Red October, The (1990)
11529	546	219	5	Nightmare on Elm Street, A (1984)
39495	457	183	5	Alien (1979)

Let's define a function to iterate over the rows of the test set and compute the related ratings:

In [19]:
```
from tqdm import tqdm
def predict(cf, test_data):
    predicted_ratings = []
    for i, ann in tqdm(test_data.iterrows(), total=len(test_data)):
        #if the user was not in the original utility matrix, let's just append
a nan
        try:
            rating = cf.predict(ann['user_id'], ann['item_id'])
        except:
            rating = np.nan
        predicted_ratings.append(rating)
    return np.array(predicted_ratings)
```

To speedup testing, let's select a few items randomly from the test set:

In [20]:
```
_, test_data_tiny = train_test_split(test_data, test_size=0.02)
len(test_data_tiny)
```

Out[20]: 501

In [21]:
```
cf3 = CollaborativeFilter(N=3)
cf3.fit(U)
predicted_ratings_3 = predict(cf3, test_data_tiny)
```

```
100%|
| 501/501 [01:45<00:00,  4.77it/s]
```

We can see recommendation as a regression problem and evaluate the system using MAE:

In [22]:
```
def mae(y_true, y_pred):
    return (y_true-y_pred).abs().mean()

mae(test_data_tiny['rating'], predicted_ratings_3)
```

Out[22]:
```
0.8396598738455379
```

Let's compare this with a collaborative filter with `N=5` :

In [23]:
```
cf5 = CollaborativeFilter(N=5)
cf5.fit(U)
predicted_ratings_5 = predict(cf5, test_data_tiny)
mae(test_data_tiny['rating'], predicted_ratings_5)
```

```
100%|
| 501/501 [01:48<00:00,  4.60it/s]
```

Out[23]:
```
0.788623628125405
```

Let's push N to some large value such as 50:

In [24]:
```
cf50 = CollaborativeFilter(N=50)
cf50.fit(U)
predicted_ratings_50 = predict(cf50, test_data_tiny)
mae(test_data_tiny['rating'], predicted_ratings_50)
```

```
100%|
| 501/501 [02:04<00:00,  4.03it/s]
```

Out[24]:
```
0.8032535120960659
```

Question 4

Why do we obtain better results for larger Ns? What is the effect of choosing a very small N?

Answer 4

If you are interested in recommendation systems, you can check the `surprise` library: [http://surpriselib.com/ (http://surpriselib.com/)](http://surpriselib.com/).

2. Word Embeddings

SpaCy provides different pre-trained word embeddings which can be used "off-the-shelf". The small model we have used does not include word embeddings, so we need to download a larger one. Specifically, SpaCy provides different models including different vocabulary sizes:

- **en_core_web_md** (https://spacy.io/models/en#en_core_web_md) (116MB) 20K embeddings
- **en_core_web_lg** (https://spacy.io/models/en#en_core_web_lg) (812MB) 685K embeddings
- **en_vectors_web_lg** (https://spacy.io/models/en#en_vectors_web_lg) (631MB) 1.1M embeddings

All Embeddings have 300 dimensions. The embeddings have not been learned using the algoritms seen in this course (GloVe). Instead they have learned using an algorithm called "word2vec". The underlying principle behind this algorithm is however the same: words which are used in a similar way have similar embeddings, also called "vectors".

We will use `en_code_web_md` , but the larger models should also been considered when building applications which strongly rely on word vectors. We can install the model with the following command:

```
python -m spacy download en_core_web_md
```

After installing the model, we can load it as follows:

In [25]:
```
import spacy
nlp = spacy.load('en_core_web_md')
doc = nlp("Is this the region, this the soil, the clime")
```

We can access the word vector with the `vector` property of each token:

In [26]:
```
v = doc[0].vector
print(len(v)) #vector dimension
v[:10] #let's show only the first 10 components of the vector
```

```
300
```

Out[26]:
```
array([-8.4961e-02,  5.0200e-01,  2.3823e-03, -1.6755e-01,  3.0721e-01,
       -2.3762e-01,  1.6069e-01, -3.6786e-01, -5.8347e-02,  2.4990e+00],
      dtype=float32)
```

2.1 Token Properties

Each token also has three additional properties:

- `is_oov` : indicates whether the word is out of vocabulary;
- `has_vector` : indicates whether the word has a word embedding;
- `vector_norm` : is the L2 norm of the word vector.

Let's see an example:

In [28]:
```
doc = nlp('Heilo how are you?')
for w in doc:
    print(w.text, "\t", w.is_oov, w.has_vector, w.vector_norm)
```

```
Heilo    True False 0.0
how      False True 5.2509694
are      False True 5.41568
you      False True 5.1979666
?        False True 5.1608233
```

The word "hello" has been mispelled as "heilo", hence it has been identified as "out of vocabulary". Since no word embedding is availble for this word, a vector containing all zeros is assigned (hence the vector norm is 0). Let's check the word vector:

In [29]:
```
doc[0].vector[:10] #first 10 components
```

Out[29]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.], dtype=float32)
```

We can obtain a word vector for a document by averaging the vectors of all the words:

In [30]:
```
import numpy as np

v = np.mean([w.vector for w in doc],0)
v[:10]
```

Out[30]:
```
array([-0.12565279,  0.1822664 , -0.23188598, -0.14474145,  0.08241   ,
       -0.090287  ,  0.09519981, -0.045398  , -0.0420998 ,  1.88006   ],
      dtype=float32)
```

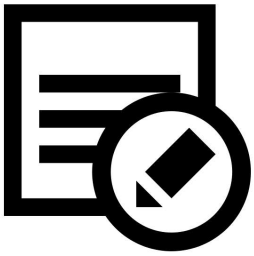
The same result can be obtained by simply calling `vector` on the document:

In [31]:
```
doc.vector[:10]
```

Out[31]:
```
array([-0.12565279,  0.1822664 , -0.23188598, -0.14474145,  0.08241   ,
       -0.090287  ,  0.09519981, -0.045398  , -0.0420998 ,  1.88006   ],
      dtype=float32)
```

Question 5

Are there any shortcomings in averaging word vectors to obtain embeddings for sentences? What happens if the sentence is very long and related to different topics?



Answer 5

2.2 Similarity

Each document/vector has a `similarity` method which allows to compute the similarity between two documents/vectors based on their word embedding:

In [32]:
```
doc = nlp("Is this the region, this the soil, the clime")
word = nlp("that")
for w in doc:
    print("{} vs {} \t {:02f}".format(word.text, w.text,w.similarity(word)))
```

```
that vs Is       0.694272
that vs this     0.672172
that vs the      0.678875
that vs region   0.343808
that vs ,        0.392222
that vs this     0.672172
that vs the      0.678875
that vs soil     0.300579
that vs ,        0.392222
that vs the      0.678875
that vs clime    0.207214
```

Similarly with documents:

In [33]:
```
doc1 = nlp("Love is real, real is love")
doc2 = nlp("All you need is love")
doc3 = nlp("Principles of Accounting is designed to meet the scope of a two-se
mester accounting course.")
print("Similarity between document 1 and 2: {:02f}".format(doc1.similarity(doc
2)))
print("Similarity between document 1 and 3: {:02f}".format(doc1.similarity(doc
3)))
```

```
Similarity between document 1 and 2: 0.873906
Similarity between document 1 and 3: 0.694209
```

2.3 Word Arithmetic

Let's see an example of arithmentic between word embeddings. For instance, let's try to see what is the closest word to:

```
"brother" - "man" + "woman"
```

In [34]:
```
from tqdm import tqdm
brother = nlp('brother')
man = nlp('man')
woman = nlp('woman')
sister = nlp('sister')

v = brother.vector - man.vector + woman.vector

cosine = lambda x,y: np.dot(x, y) / (np.sqrt(np.dot(x,x)) * np.sqrt(np.dot(y,
y)))

words = []
similarities = []
for word in tqdm(nlp.vocab, total=len(nlp.vocab)):
    if word.has_vector:
        if word.is_lower:
            words.append(word.text)
            similarities.append(cosine(v, word.vector))
np.array(words)[np.argsort(similarities)[::-1]]
```

```
100%|
| 518/518 [00:00<00:00, 104024.20it/s]
```

Out[34]:
```
array(['sister', 'brother', 'woman', 'she', 'who', 'when', 'had', 'was',
       'love', 'he', 'got', 'did', "'m", 'man', 'meet', 'two', 'let', 'a',
       'would', 'why', 'where', 'could', 'dare', "'d", 'b.', 'does', 'ca',
       'how', 'that', 'there', "n't", 'n’t', 'what', 'have', 'not', 'i',
       "'s", '’s', 'they', 'has', '’ve', "'ve", 'you', 'might', 'were',
       'd.', 'to', 'am', 'it', 'cause', 'is', 'real', 'the', 'do', 'wo',
       "'ll", '’ll', "'re", '’re', 'this', "ma'am", 'those', 'should',
       's', "'cause", 'ought', 'lovin', 'may', 'must', 'of', 'we',
       'course', 'all', 'can', 'goin', 'i.', 'o’clock', "o'clock", 'a.m.',
       'havin', 'need', 'm.', "y'", 'a.', 'nuff', 'on', 'y', 've', 'ai',
       'doin', 's.', 'w.', 'p.m.', 'these', 'll', 'ol', 'and/or', 'are',
       'u.', 'semester', 'co.', ':p', 'nothin', 'somethin', 'nuthin',
       'region', 'w/o', 'gon', 'r.', 'y.', 're', 'designed', 'c.', 'd',
       'i.e.', 'nt', 'o.', 't.', 'o.o', 'm', 'x.', 'em', 'accounting',
       'pm', 'z.', 'k.', 'sha', 'ta', 'l.', 'clime', 'na', 'g.', 'f.',
       'v.', 'e.', '0_o', 'o_o', 'v.v', 'o_0', 'h.', 'j.', "'em", '’em',
       'vs.', 'p.', 'n.', ':-p', 'q.', 'soil', 'e.g.', 'v_v', 'scope',
       '’cause', 'v.s.'], dtype='<U10')
```

As one could expect, the closest word is "sister".

3. Sentiment Analysis

We will now see how to perform sentiment anlysis on a text using VADER. We will use a module included in the NLTK library. First, we need to download the VADER lexicon as follows:

In [35]:
```
import nltk
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package vader_lexicon to
[nltk_data]     C:\Users\Frà\AppData\Roaming\nltk_data...
```

Out[35]:
```
True
```

We will use the `SentimentIntensityAnalyzer` object:

In [36]:
```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
sid = SentimentIntensityAnalyzer()
```

To obtain the `negative`, `neutral`, `positive` and `compound` scores of VADER, we will use the `polarity_scores` method:

In [37]:
```
sid.polarity_scores("The movie was great.")
```

Out[37]:
```
{'neg': 0.0, 'neu': 0.423, 'pos': 0.577, 'compound': 0.6249}
```

This will return a dictionary containing the four scores. Let's try with another few examples:

In [38]:
```
sid.polarity_scores("The movie was awful.")
```

Out[38]:
```
{'neg': 0.5, 'neu': 0.5, 'pos': 0.0, 'compound': -0.4588}
```

In [39]:
```
sid.polarity_scores("The movie was more than GREAT!")
```

Out[39]:
```
{'neg': 0.0, 'neu': 0.481, 'pos': 0.519, 'compound': 0.7509}
```

In [40]:
```
sid.polarity_scores("The movie was OK")
```

Out[40]:
```
{'neg': 0.0, 'neu': 0.506, 'pos': 0.494, 'compound': 0.4466}
```

Question 7

Compare the third example to the first one. Why does the third one have a larger compound value than the first one?

Answer 7

3.1 Sentiment Analysis and Movie Reviews

Let's now see how we can use VADER to analyze movie reviews. We will use the movie reviews dataset seen in the previous laboraties:

In [41]:
```
import pandas as pd
reviews=pd.read_csv('https://iplab.dmi.unict.it/ragusa/downloads/reviews.csv')
reviews.info()
reviews.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5006 entries, 0 to 5005
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   author  5006 non-null   object
 1   review  5006 non-null   object
 2   rating  5006 non-null   float64
dtypes: float64(1), object(2)
memory usage: 117.5+ KB
```

Out[41]:

	author	review	rating
0	Dennis_Schwartz	in my opinion , a movie reviewer's most import...	0.1
1	Dennis_Schwartz	you can watch this movie , that is based on a ...	0.2
2	Dennis_Schwartz	this is asking a lot to believe , and though i...	0.2
3	Dennis_Schwartz	no heroes and no story are the main attributes...	0.2
4	Dennis_Schwartz	this is not an art movie , yet i saw it an art...	0.2

Let's analyze the first review with VADER:

In [42]:
```
sid.polarity_scores(reviews.loc[0]['review'])
```

Out[42]:
```
{'neg': 0.134, 'neu': 0.753, 'pos': 0.113, 'compound': -0.8923}
```

The "compound" score is negative, which is coherent with low rating assigned to the review. Let's see if this happens systematically. To do so, we need to compute the "compound" value for each review. We will first define a `vader_polarity` function to compute the polarity of the review using the compound:

In [43]:
```
vader_polarity = lambda x: sid.polarity_scores(x)['compound']
vader_polarity(reviews.loc[0]['review'])
```

Out[43]:
```
-0.8923
```

Let's now compute the polarity score to each review (this might take a while):

In [44]:
```
tqdm.pandas()
reviews['polarity']=reviews['review'].progress_apply(vader_polarity)
reviews.head()
```

```
100%|
| 5006/5006 [00:30<00:00, 163.70it/s]
```

Out[44]:

	author	review	rating	polarity
0	Dennis_Schwartz	in my opinion , a movie reviewer's most import...	0.1	-0.8923
1	Dennis_Schwartz	you can watch this movie , that is based on a ...	0.2	0.8927
2	Dennis_Schwartz	this is asking a lot to believe , and though i...	0.2	0.9772
3	Dennis_Schwartz	no heroes and no story are the main attributes...	0.2	0.0316
4	Dennis_Schwartz	this is not an art movie , yet i saw it an art...	0.2	0.9904

3.2 Classifying reviews with no training

To see if the polarity tells us something about the rating, let's consider a binary classification task. We will consider a review as negative if the rating is smaller than 0.5. Similarly, we will classify a review as positive if the polarity is positive:

In [45]:
```
reviews['label']=reviews['rating']>=0.5
reviews['predicted_label']=reviews['polarity']>0
reviews.head()
```

Out[45]:

	author	review	rating	polarity	label	predicted_label
0	Dennis_Schwartz	in my opinion , a movie reviewer's most import...	0.1	-0.8923	False	False
1	Dennis_Schwartz	you can watch this movie , that is based on a ...	0.2	0.8927	False	True
2	Dennis_Schwartz	this is asking a lot to believe , and though i...	0.2	0.9772	False	True
3	Dennis_Schwartz	no heroes and no story are the main attributes...	0.2	0.0316	False	True
4	Dennis_Schwartz	this is not an art movie , yet i saw it an art...	0.2	0.9904	False	True

Let's now assess the performance of our classifier:

In [46]:
```
from sklearn.metrics import accuracy_score, confusion_matrix
print("Accuracy: {:0.2f}%".format(accuracy_score(reviews['label'], reviews['pr
edicted_label'])*100))
confusion_matrix(reviews['label'], reviews['predicted_label'])
```

```
Accuracy: 73.93%
```

Out[46]:
```
array([[ 437,  820],
       [ 485, 3264]], dtype=int64)
```

The model is not perfect, but the result is not bad, considering that we have not trained at model at all!

Question 8

How does this model compares with respect to the model based on bag of words seen in the previous laboratories?

Answer 8

3.3 VADER Scores as Features

We can do something slightly more complicated by considering all VADER scores as features and training a logistic regressor on top of them. We did not cover logistic regression during this course, but think about it as a type of classifier. We should now define a function that maps a review to a feature vector containing the VADER scores:

In [53]:
```
vader_features = lambda x: np.array(list(sid.polarity_scores(x).values()))
vader_features(reviews.iloc[0]['review'])
```

Out[53]:
```
array([ 0.134 ,  0.753 ,  0.113 , -0.8923])
```

We will use a very small training set to see what we can done with very little training.

In [54]:
```
from sklearn.model_selection import train_test_split
np.random.seed(123)
reviews_train, reviews_test = train_test_split(reviews, test_size=0.99) #let's
use only 1% of the data for training
print(len(reviews_train))
```

```
50
```

Let's compute a feature vector for each review of the training and test sets:

In [49]:
```
x_train = np.vstack(list(reviews_train['review'].progress_apply(vader_feature
s)))
x_test = np.vstack(list(reviews_test['review'].progress_apply(vader_feature
s)))
```

```
100%|
| 50/50 [00:00<00:00, 146.03it/s]
100%|
| 4956/4956 [00:30<00:00, 164.84it/s]
```

Let's obtain the related labels:

In [50]:
```
y_train = reviews_train['label']
y_test = reviews_test['label']
```

Let's now train the logistic regressor. We will normalize features using a MinMaxScaler:

In [51]:
```
from sklearn.linear_model import LogisticRegression
reg = LogisticRegression()
reg.fit(x_train, y_train)
reg.score(x_test, y_test)
```

Out[51]:
```
0.7487893462469734
```

Using only 50 reviews for training we have obtained a small boost on performance. We can also try to fit a linear regressor to predict the ratings:

In [52]:
```
from sklearn.linear_model import LinearRegression
y_train = reviews_train['rating']
y_test = reviews_test['rating']

reg = LinearRegression()
reg.fit(x_train, y_train)
y_test_pred = reg.predict(x_test)
mae(y_test, y_test_pred)
```

Out[52]:
```
0.14117188504941192
```

We have obtained a relatively small MAE error, considering the range of the reviews.

References

[1] Recommendation systems library. http://surpriselib.com/ (http://surpriselib.com/)

[2] Word vectors in Spacy. https://spacy.io/usage/vectors-similarity (https://spacy.io/usage/vectors-similarity)

[3] NLTK and VADER. https://www.nltk.org/_modules/nltk/sentiment/vader.html (https://www.nltk.org/_modules/nltk/sentiment/vader.html)

Exercises

Exercise 1

Build an item-item Collaborative Filter and compare its performance with respect to the user-user Collaborative Filter built in this laboratory. Experiment with different values of `N`. Which of the two approaches achieves the best performance?

Exercise 2

Represent each review of the movie review dataset using the word embeddings. Use a small portion of the data to train a logistic regressor for review classification on the dataset. How does this model compares with respect to the one built using VADER words embeddings?

Exercise 3

Consider the ham vs spam dataset seen in the previous laboratories. Represent each element using features extracted with VADER and word embeddings. Train two logistic regressors using the two set of features using a small portion of the data as training set. Compare the two models with a logistic regressor built starting from a bag of words representation. Use the same train/test split for training/evaluation. Which of the models performs best? Why?